=== UNIT 2: CONTROL FLOW, CONDITIONAL LOGIC & ITERATION ===

>> 2.1 Conditional Decision Making in Python

Control flow constructs govern the non-linear execution path of computer programs based on dynamic runtime conditions.
* Syntax of Decision Structures: if condition: statement; elif alternative: statement; else: fallback_statement.
* Truthiness and Falsy Values: In Python, conditions evaluate according to boolean truthiness. The following evaluate to False: (1) None, (2) False, (3) Zero numeric values (0, 0.0, 0j), (4) Empty sequence and collection containers (’’, (), [], {}, set()), (5) Custom objects implementing __bool__() returning False or __len__() returning 0. All other values evaluate to True.
* Conditional Ternary Expressions: Single-line conditional assignment: x = true_value if condition else false_value.
* Structural Pattern Matching (match-case, introduced in Python 3.10): Provides expressive pattern decomposition and guard filtering similar to switch-case statements in C.

>> 2.2 While Loops and Definite Iteration with For Loops

* The While Loop: Executes a code block repeatedly as long as a test condition remains truthy: while condition: body.
- Infinite Loops: Caused by failing to update loop control variables inside the body (while True:).
* The For Loop: Python’s for loop is an iterator-driven ’for-each’ construct that traverses items in any iterable object (list, tuple, string, dictionary, generator): for item in iterable: body.
* The range() Built-in Function: Generates an immutable arithmetic sequence of integers on-demand (generator-like memory efficiency): range(start, stop[, step]). Memory consumption is O(1) regardless of sequence length.

>> 2.3 Loop Interruption and Control Statements

* break Statement: Immediately terminates the innermost loop, transferring control to the statement following the loop body.
* continue Statement: Aborts execution of the remainder of the current loop iteration and jumps immediately to the next iteration cycle.

* pass Statement: A null operation (syntactic placeholder) where Python syntax requires a statement but no programmatic action is desired.
* The Loop else Clause: A unique Python feature: both for and while loops can have an optional else clause. The else block executes ONLY if the loop completes all normal iterations WITHOUT encountering a break statement. Commonly used in search algorithms.

>> 2.4 Nested Loops and Iteration Utilities

* Nested Loops: Loops enclosed inside another loop (e.g. traversing 2D matrices). Computational complexity scales as O(n * m).
* enumerate(iterable, start=0): Yields tuples containing (index, item) during iteration, eliminating manual counter management.
* zip(*iterables): Aggregates elements from multiple iterables in parallel into tuples, stopping when the shortest iterable is exhausted.

[PROGRAM 1: Prime Number Generation with Loop Else Clause]

Problem Context: Write an optimized program finding all prime numbers in a user-defined range [start, end]. Use trial division up to sqrt(n) and utilize Python’s loop else construct to detect primes cleanly without boolean flags.

Python Source Code:

```
import math

def find_primes(start, end):
primes = []
for num in range(max(2, start), end + 1):
limit = int(math.isqrt(num))
for divisor in range(2, limit + 1):
if num % divisor == 0:
break  # Composite number detected; abort search
else:
# Loop completed without break => num is prime!
primes.append(num)
return primes

results = find_primes(10, 50)
print(f’Primes in [10, 50]: {results}’)
print(f’Total primes found: {len(results)}’)
```

Execution Trace & Output:
Primes found: [11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]. Total count = 11 primes.

[PROGRAM 2: Fibonacci Sequence Generator via While Loop]

Problem Context: Implement a program generating the first N terms of the Fibonacci sequence using a while loop and simultaneous variable assignment.
Python Source Code:

```
def generate_fibonacci(n):
if n <= 0:
return []
fib = []
a, b = 0, 1
count = 0
while count < n:
fib.append(a)
a, b = b, a + b  # Simultaneous tuple unpacking
count += 1
return fib

terms = 10
fib_series = generate_fibonacci(terms)
print(f'First {terms} Fibonacci numbers: {fib_series}')
```

Execution Trace & Output:
Iterations: count 0: [0], a=1,b=1; count 1: [0, 1], a=1,b=2; count 2: [0, 1, 1], a=2,b=3...
Final output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34].

[PROGRAM 3: Pascal's Triangle Matrix Generation using Nested Loops]

Problem Context: Construct Pascal's Triangle of depth N rows using nested for loops and list representation.
Python Source Code:

```
def pascals_triangle(n):
triangle = []
for row_idx in range(n):
row = [1] * (row_idx + 1)
for col_idx in range(1, row_idx):
```

```
row[col_idx] = triangle[row_idx - 1][col_idx - 1] + triangle[row_idx - 1][col_idx]
triangle.append(row)
return triangle

n_rows = 5
triangle = pascals_triangle(n_rows)
for r in triangle:
print(' '.join(f'{x:3d}' for x in r).center(30))
```

Execution Trace & Output:
Row 0: [1]
Row 1: [1, 1]
Row 2: [1, 2, 1]
Row 3: [1, 3, 3, 1]
Row 4: [1, 4, 6, 4, 1]

[PROGRAM 4: Parallel Sequence Traversal with Enumerate and Zip]

Problem Context: Given separate lists of student names, marks in Python, and attendance percentages, calculate grades and identify students eligible for academic honors (marks >= 85 and attendance >= 80%).
Python Source Code:

```
students = ['Aarav', 'Diya', 'Rohan', 'Ananya']
marks = [92.0, 78.5, 88.0, 64.0]
attendance = [95.0, 85.0, 72.0, 90.0]

print('Rank | Student Name | Marks | Attend | Status')
print('-' * 52)
for rank, (name, m, att) in enumerate(zip(students, marks, attendance), start=1):
is_honors = (m >= 85.0) and (att >= 80.0)
status = 'HONORS' if is_honors else 'PASS'
print(f'{rank:4d} | {name:12s} | {m:5.1f} | {att:5.1f}% | {status}')
```

Execution Trace & Output:
Aarav (92.0, 95%) -> HONORS; Diya (78.5, 85%) -> PASS; Rohan (88.0, 72%) -> PASS (attendance below 80%); Ananya (64.0) -> PASS.

>> 2.5 University Review & Examination Questions

* Q1: Explain the semantics of the loop 'else' clause in Python and construct an example demonstrating its practical utility.

Answer: In Python, both ’for’ and ’while’ loops support an optional ’else’ block. The ’else’ suite executes if and only if the loop terminates normally by exhausting the iterable sequence or when the while condition becomes False. If the loop is terminated prematurely by a ’break’ statement, the ’else’ block is completely bypassed. This provides an elegant mechanism for search operations without requiring auxiliary flag variables: in searching for an element in a list, if a matching item is found, the program executes ’break’; if the search finishes without finding the item, the ’else’ block executes to handle the ’item not found’ scenario.

* Q2: Differentiate between ’break’, ’continue’, and ’pass’ statements with syntax flowcharts.

Answer: ’break’ terminates the loop execution immediately, transferring control to the statement following the loop. ’continue’ skips all subsequent code in the current iteration and jumps immediately to the next evaluation cycle (incrementing iterator or checking condition). ’pass’ is a null syntactic statement that does nothing; it acts as a placeholder where Python syntax demands an indented statement block (e.g., in empty function stubs, abstract methods, or exception pass-throughs).

>> 2.6 Key Takeaways and Revision Points

* Falsy values include None, False, 0, and empty collections ([], {}, ’’).
* range(start, stop, step) evaluates lazily with O(1) memory overhead.
* Loop ’else’ executes only when the loop completes without encountering a break.
* zip() and enumerate() facilitate clean, idiomatic multi-sequence iteration.